

Large Language Models: From Tokens to Generation

Для студентов, знакомых с PyTorch

План презентации

1. **Токенизация** - как текст превращается в числа
2. **Embeddings & Positional Encoding** - представление слов
3. **Transformer & Attention** - архитектура и механизм внимания
4. **Обучение LLM** - pre-training и fine-tuning
5. **Генерация текста** - sampling стратегии

1. Токенизация

Зачем нужна токенизация?

Нейросети работают с числами, а не с текстом. Токенизация разбивает текст на **токены** (подслова/символы).

Подходы:

- **Word-based**: каждое слово → токен (проблема: большой словарь)
- **Character-based**: каждый символ → токен (проблема: длинные последовательности)
- **Subword**: компромисс (Byte-Pair Encoding, WordPiece, Unigram)

Byte-Pair Encoding (BPE): Математика

Алгоритм обучения:

Инициализация: $V_0 = \{c_1, c_2, \dots, c_n\}$ - алфавит символов

Итеративный процесс (повторять k раз):

1. Подсчитать частоты пар:

$$f(a, b) = \sum_{w \in C} \text{count}(a \cdot b \text{ в } w) \cdot \text{freq}(w)$$

2. Выбрать наиболее частую пару:

$$(a^*, b^*) = \arg \max_{(a, b)} f(a, b)$$

3. Объединить: $V_{i+1} = V_i \cup \{a^*b^*\}$

4. Заменить все вхождения $a^*b^* \rightarrow a^*b^*$ в корпусе

Результат: Словарь V_k размера $|V_0| + k$

ВРЕ: Пример

Исходный корпус:

- "low" (частота 5)- "lower" (частота 2) "newest" (частота 6) "widest" (частота 3)

Начальный словарь: $\{w, e, s, t, l, o, w, n, i, d, r\}$

Итерация 1:

- Частые пары: $(e, s) = 9$, $(s, t) = 9$
- Выбираем $(e, s) \rightarrow es$
- Новый словарь: $\{w, e, s, t, l, o, n, i, d, r, es\}$

Итерация 2:

- Частые пары: $(es, t) = 9$
- Выбираем $(es, t) \rightarrow est$
- "newest" $\rightarrow$ "n ew est"

После 10 итераций: "lowest" $\rightarrow$ "l o w est"

Токенизатор в действии

Пример использования:

```
Текст: "Привет, мир!"  
↓ Токенизация  
Токены: ['Пр', 'ив', 'ет', ',', ' ', 'мир', '!']  
↓ Конвертация  
IDs: [12520, 4325, 2856, 11, 7039, 0]
```

Специальные токены:

- **<PAD>** - padding (дополнение до фикс. длины)
- **<UNK>** - неизвестные токены
- **<BOS>** / **<EOS>** - начало/конец последовательности
- **<SEP>** - разделение сегментов

Размер словаря: 30,000 - 100,000 токенов

2. Embedding Layer

Что такое embedding?

Embedding - проекция дискретного токена в непрерывное векторное пространство.

Математически:

$$\mathbf{E} : V \rightarrow \mathbb{R}^d$$

где:

- V - словарь, $|V| = \text{vocab_size}$
- d - размерность embedding (768, 1024, ...)

Операция:

$$\mathbf{x}_{emb} = \mathbf{W}_E[\mathbf{x}_{token}]$$

где $\mathbf{W}_E \in \mathbb{R}^{|V| \times d}$ - матрица embeddings

Пример:

- $\text{vocab_size} = 50,000$
- $d = 768$
- $\mathbf{W}_E \in \mathbb{R}^{50000 \times 768}$

Positional Encoding

Проблема:

Transformer **не имеет понятия порядка** токенов (в отличие от RNN).

Решение:

Добавить **позиционную информацию** к embeddings:

$$\mathbf{X}_{pos} = \mathbf{E}_{token} + \mathbf{P}_{pos}$$

Sinusoidal encoding (Vaswani et al., 2017):

$$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{2i/d_{model}}} \right)$$

$$PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{2i/d_{model}}} \right)$$

где:

- pos - позиция токена $(0, 1, \dots, L - 1)$
- i - индекс размерности $(0, 1, \dots, d_{model}/2 - 1)$

Преимущество: модель может экстраполировать на последовательности длиннее, чем видела при обучении.

Positional Encoding: визуализация

Свойства:

1. **Уникальность:** Каждая позиция имеет уникальный паттерн
2. **Относительность:** PE_{pos+k} может быть представлена как линейная функция PE_{pos}
3. **Экстраполяция:** Работает на длинах, не виденных при обучении

Пример вычисления (для $d_{model} = 4$):

Позиция	PE[0]	PE[1]	PE[2]	PE[3]
0	$\sin(0)$	$\cos(0)$	$\sin(0)$	$\cos(0)$
1	$\sin(0.01)$	$\cos(0.01)$	$\sin(0.0001)$	$\cos(0.0001)$
2	$\sin(0.02)$	$\cos(0.02)$	$\sin(0.0002)$	$\cos(0.0002)$

3. Transformer Architecture

Общая архитектура

Encoder-Decoder структура:

- **Encoder:** обрабатывает входную последовательность
- **Decoder:** генерирует выходную последовательность

LLM используют только Decoder! (GPT-архитектура)

Основные компоненты:

- Multi-Head Attention
- Feed-Forward Networks
- Layer Normalization
- Residual Connections

Attention Mechanism

Attention как Key-Value база данных

Аналогия с поиском в БД:

- **Query (Q):** ваш запрос ("найти информацию о кошках")
- **Keys (K):** индексы/теги в базе ("собаки", "кошки", "птицы")
- **Values (V):** сами данные (статьи о животных)

Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

где:

- $Q \in \mathbb{R}^{n \times d_k}$ - запросы
- $K \in \mathbb{R}^{m \times d_k}$ - ключи
- $V \in \mathbb{R}^{m \times d_v}$ - значения
- d_k - размерность key/query

Масштабирование $\frac{1}{\sqrt{d_k}}$ предотвращает исчезновение градиентов при больших d_k

Как работает Attention: Пример

- Предложение: "The cat sat on the mat"
- Для токена "cat" вычисляем attention

Шаги:

1. Вычисление сходства:

$$\text{score}(q_{cat}, k_i) = q_{cat} \cdot k_i^T$$

Для каждого токена i получаем:

- "The": 0.5
- "cat": 2.1 (само с собой)
- "sat": 1.3
- "on": 0.2
- "the": 0.6
- "mat": 1.8

2. **Softmax** (нормализация):

$$\alpha_i = \frac{\exp(\text{score}_i)}{\sum_j \exp(\text{score}_j)}$$

Веса: [0.05, 0.25, 0.12, 0.03, 0.06, 0.20]

3. Взвешенная сумма:

$$\text{output} = \sum_i \alpha_i v_i$$

Multi-Head Attention

Идея:

Один attention механизм → недостаточно. Нужно смотреть на разные **аспекты** одновременно.

Математика:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

где каждый head:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

и:

- $W_i^Q \in \mathbb{R}^{d_{model} \times d_k}$
- $W_i^K \in \mathbb{R}^{d_{model} \times d_k}$
- $W_i^V \in \mathbb{R}^{d_{model} \times d_v}$
- $W^O \in \mathbb{R}^{hd_v \times d_{model}}$

Пример: $d_{model} = 512$, $h = 8$ голов $\Rightarrow d_k = d_v = 64$

Multi-Head Attention: интуиция

Разные головы учатся разному:

- **Head 1:** Синтаксические связи (подлежащее → сказуемое)
- **Head 2:** Корелляция (местоимения → существительные)
- **Head 3:** Семантическая близость (синонимы)
- **Head 4:** Позиционные паттерны
- **Head 5-8:** Другие лингвистические закономерности

Пример для "it" в "The animal didn't cross because it was tired":

- Head 2: высокий вес на "animal"
- Head 3: средний вес на "tired"

Transformer Decoder Block

Компоненты:

1. **Masked Multi-Head Attention:** $\text{MHA}(Q, K, V)$ с маской

2. **Feed-Forward Network (MLP):**

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

где $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$

3. **LayerNorm:** $\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta$

4. **Residual Connections:** $x + \text{Sublayer}(x)$

Masked Attention

Почему нужна маска?

При генерации мы не можем видеть будущие токены!

Математика маскирования:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V$$

где маска $M \in \mathbb{R}^{n \times n}$:

$$M_{ij} = \begin{cases} 0 & \text{если } j \leq i \\ -\infty & \text{если } j > i \end{cases}$$

Пример для $n = 4$:

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Результат: $\text{softmax}(-\infty) = 0$, токены справа не влияют

4. Обучение LLM

Pre-training: Language Modeling

Задача: предсказать следующий токен

Функция потерь (Cross-Entropy):

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log P(x_t | x_{<t}; \theta)$$

или эквивалентно:

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{i=1}^{|V|} y_{t,i} \log \hat{y}_{t,i}$$

где:

- $y_{t,i}$ - one-hot encoding истинного токена
- $\hat{y}_{t,i} = \text{softmax}(z_{t,i})$ - предсказанная вероятность

Пример:

- Истинная последовательность: ["The", "cat", "sat"]
- Предсказания модели: $P(\text{"cat"} | \text{"The"}) = 0.7$
- $\mathcal{L} = -\log(0.7) \approx 0.357$

Training Data & Scale

Данные для обучения:

- **Common Crawl** - веб-страницы (терабайты текста)
- **Books** - книги (литература, нон-фикшн)
- **Wikipedia** - энциклопедические статьи
- **Code** - GitHub (программный код)
- **News** - новостные статьи

Масштаб современных LLM:

Модель	Параметры	Данные	GPU-часы
GPT-2	1.5B	40GB	~10K
GPT-3	175B	45TB	~3.6M
LLaMA 7B	7B	1.4Т токенов	~180K
LLaMA 65B	65B	1.4Т токенов	~1.2M

Закон масштабирования (Kaplan et al., 2020):

$$L(N, D) \approx \left(\frac{N_c}{N} \right)^{\alpha_N} + \left(\frac{D_c}{D} \right)^{\alpha_D} + L_0$$

где N - параметры, D - данные, $\alpha_N \approx 0.34$, $\alpha_D \approx 0.28$

Fine-tuning

После pre-training:

1. Supervised Fine-Tuning (SFT):

Обучаем на размеченных данных (инструкция → ответ):

$$\mathcal{L}_{SFT}(\theta) = - \sum_{(x,y) \in D_{SFT}} \log P(y|x; \theta)$$

2. RLHF (Reinforcement Learning from Human Feedback):

- **Шаг 1:** Собираем human preferences $D_{pref} = \{(x, y_w, y_l)\}$
где y_w - preferred, y_l - less preferred
- **Шаг 2:** Обучаем reward model:

$$\mathcal{L}_{RM}(\phi) = -\mathbb{E}_{(x,y_w,y_l)} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

- **Шаг 3:** Оптимизируем policy (PPO):

$$\mathcal{L}_{PPO}(\theta) = \mathbb{E}[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$$

5. Генерация текста

Greedy Decoding

Алгоритм:

$$x_t = \arg \max_x P(x|x_{<t})$$

Пример:

Шаг	Контекст	Распределение P(x context)	Выбор
1	"<BOS>"	The: 0.4, A: 0.3, Once: 0.2, ...	"The"
2	"<BOS> The"	cat: 0.5, dog: 0.3, quick: 0.1, ...	"cat"
3	"<BOS> The cat"	sat: 0.6, jumped: 0.2, sleeps: 0.1, ...	"sat"

Результат: "The cat sat"

Проблема: Повторяющийся, скучный текст (мало разнообразия)

Temperature Sampling

$$P_T(x_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

где:

- z_i - logits модели
- T - температура ($T > 0$)

Эффекты:

- $T < 1$: Более "уверенное" распределение (пики выше)
 - $T \rightarrow 0$: $\Rightarrow$ Greedy decoding
 - Пример: $T = 0.5 \rightarrow$ более детерминированный текст
- $T = 1$: Исходное распределение
- $T > 1$: Более "плоское" распределение (выше энтропия)
 - Пример: $T = 1.5 \rightarrow$ более креативный, случайный текст

Пример (logits: [2.0, 1.0, 0.5]):

T	P(x ₁)	P(x ₂)	P(x ₃)
0.5	0.84	0.14	0.02
1.0	0.60	0.24	0.16
2.0	0.42	0.31	0.27

Top-K Sampling

Алгоритм:

1. Выбрать K наиболее вероятных токенов:

$$V_K = \text{TopK}(V, K)$$

2. Перераспределить вероятности:

$$P'(x_i) = \begin{cases} \frac{P(x_i)}{\sum_{x_j \in V_K} P(x_j)} & \text{если } x_i \in V_K \\ 0 & \text{иначе} \end{cases}$$

3. Сэмплировать из P'

Пример (K=3):

Токен	Исходная P	После Top-3	Нормализованная P'
"the"	0.40	0.40	0.50
"cat"	0.25	0.25	0.31
"dog"	0.15	0.15	0.19
"bird"	0.10	0.00	0.00
"fish"	0.10	0.00	0.00

Преимущество: Отсекает маловероятные, бессмысленные токены

Top-p (Nucleus) Sampling

Идея:

Выбираем **минимальное множество** токенов, чья суммарная вероятность $\geq p$

Алгоритм:

1. Сортируем токены по убыванию вероятности: $P(x_1) \geq P(x_2) \geq \dots$
2. Находим минимальное k :

$$V_p = \{x_1, \dots, x_k\}, \quad \text{где} \quad \sum_{i=1}^k P(x_i) \geq p$$

3. Перераспределяем:

$$P'(x_i) = \begin{cases} \frac{P(x_i)}{\sum_{x_j \in V_p} P(x_j)} & \text{если } x_i \in V_p \\ 0 & \text{иначе} \end{cases}$$

Пример ($p=0.9$):

Токен	P	Cumulative Sum	Включить?
"the"	0.40	0.40	✓
"cat"	0.25	0.65	✓
"dog"	0.15	0.80	✓
"sat"	0.12	0.92	✓ (≥ 0.9)
"i"	0.08	1.00	✗

Сравнение методов генерации

Метод	Формула	Качество	Разнообразие	Скорость
Greedy	$\arg \max P(x)$	Хорошо	Низкое	Быстро
Temperature	$P_T \propto \exp(z/T)$	Хорошо	Среднее	Быстро
Top-K	$P' \propto P$ на Top-K	Хорошо	Высокое	Быстро
Top-p	$P' \propto P$ на Nucleus	Отлично	Высокое	Быстро
Beam Search	$\max \prod P(x_t)$	Отлично	Низкое	Медленно

Рекомендации:

Креативные задачи (стихи, истории):

- Top-p ($p=0.9$), $T=0.7-0.9$

Фактические задачи (ответы на вопросы):

- Top-p ($p=0.95$), $T=0.1-0.3$

Код:

- Greedy или низкая температура ($T=0.2$)

Практический пример генерации

Вход: "Once upon a time"

Шаг 1: Токенизация

- Tokens: ["Once", "Gupon", "Ga", "Gtime"]
- IDs: [3845, 421, 257, 1234]

Шаг 2: Forward pass

- Получаем logits для последнего токена
- Применяем softmax: $P(x|\text{context})$

Шаг 3: Sampling (Top-p=0.9, T=0.8)

- Сортируем токены
- Выбираем nucleus
- Сэмплируем: "," (запятая)

Шаг 4: Итерация

- Новый контекст: "Once upon a time,"
- Повторяем шаги 2-3

Результат после 10 итераций:

"Once upon a time, there was a brave knight who lived in a small village..."

Ключевые концепции

1. Токенизация

- BPE: итеративное объединение частых пар
- Размер словаря: 30K-100K токенов

2. Embeddings + Positional Encoding

- $\mathbf{E} : V \rightarrow \mathbb{R}^d$
- $PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$

3. Attention

- $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
- Multi-head: h параллельных "представлений"

4. Обучение

- Pre-training: $\mathcal{L} = -\sum \log P(x_t | x_{<t})$
- Fine-tuning: SFT + RLHF

5. Генерация

- Temperature: $P_T \propto \exp(z/T)$
- Top-p: nucleus с вероятностью p

Дополнительные ресурсы

Документация:

-  [Hugging Face Course](#)
-  [The Illustrated Transformer](#)
-  [Attention Is All You Need](#)

Код:

-  [Transformers library](#)
-  [NanoGPT](#)

Практика:

-  [Hugging Face Hub](#)
-  [Fine-tuning tutorials](#)

Вопросы?

Спасибо за внимание!

Бэкип: Основные формулы

Layer Normalization

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta$$

$$\text{где } \mu = \frac{1}{d} \sum x_i, \sigma^2 = \frac{1}{d} \sum (x_i - \mu)^2$$

Position-wise Feed-Forward

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

$$\text{где } W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}, W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$$

Residual Connection

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

Cross-Entropy Loss

$$\mathcal{L} = - \sum_{i=1}^{|V|} y_i \log(\hat{y}_i)$$

где y - true distribution (one-hot), $\hat{y} = \text{softmax}(z)$